School Canteen Inventory Monitoring System

Introduction To Computing Project

Presented to the
Faculty of the College of Information Technology
Cagayan de Oro College
Cagayan de Oro City

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Science in Information Technology

NAME OF RESEARCHER
SALAGANTIN, GERIECE L.
TOLENTINO, CLARK HAROLD JOHN
ONDAY, ALDREN
AMOLATO, JOHN CARLO S.
DABA, NATHANIEL
PICSON, KATHLYN C.

OCTOBER, 2026

1.1 Introduction

Inventory monitoring helps organizations keep track of the quantity and status of the items they sell or use. In a school canteen, where food, drinks, ingredients, and disposables are consumed every day, knowing what is on hand is necessary to avoid running out of popular items and to decide when to restock. Recording stock in a consistent way also lets the people in charge see which items need attention. This is relevant to canteen operations because small shortages or recording errors can quickly affect the service given to students and staff. Differences between recorded and actual stock, known as inventory record inaccuracy, have been linked to weaker store-level performance (Shabani et al., 2021).

For this project, the proponents assume a school canteen that records its stock manually and handles deliveries and daily sales by hand. No actual client interviews or records were collected, so the following are possible difficulties that the project considers and not verified findings. First, items that are low or already out of stock may go unnoticed until a customer asks for them. Second, stock changes from deliveries and sales may be recorded inconsistently, and a single fixed low stock number does not fit items that are stocked in very different amounts. Third, when access is not controlled, anyone using the records could change quantities or accounts, so the staff who handle daily work are not separated from the manager in charge of the inventory. A study of a school cafeteria likewise reported product shortages and inaccurate information caused by manual transactions and the lack of a proper inventory system (Sison et al., 2019).

The proponents developed the School Canteen Inventory Monitoring System using a Python console-based application to help canteen managers and staff keep track of stock. The system allows users to log in, view the stock list, record deliveries and items sold or used, and view a low stock report. The manager can also add and delete items, adjust stock by physical count, view an inventory summary report, and manage staff accounts. These functions address the three problems by labeling each item as In Stock, Low Stock, or Out of Stock with a suggested restock amount, by validating every stock entry and treating the quantity entered for a new item as its full stock so that the low stock alert comes at 20% of that full stock, and by showing each user only the menu options allowed for their role.

1.2 Statement of the Problem

General Problem

In a school canteen that tracks its stock manually, the manager and staff may find it difficult to know the current quantity of each item, which items need restocking, and who is allowed to change the records. As a result, items may run out unexpectedly, recorded quantities may become inaccurate, and daily canteen operations may be disrupted.

Specific Problems

1. Low and out-of-stock items may go unnoticed. Without a regular check, staff may only learn that an item is gone when a customer asks for it, which delays restocking.
2. Stock changes may be recorded inconsistently. Deliveries and sales may be written down differently, and one fixed low stock number does not suit items stocked in different amounts, which makes the recorded quantity and its status unreliable.
3. Records and accounts are not protected by roles. When every user can change every record, daily staff may alter quantities or accounts that only the manager should control.

1.3 Objectives of the Study

General Objective

To develop the School Canteen Inventory Monitoring System, a Python console-based application that enables canteen managers and staff to monitor item stock, record stock changes, and access features according to their role.

Specific Objectives

1. To compute and display each item’s stock status based on its own full stock, with low stock alerts at 20% of the full stock and suggested restock amounts.
2. To record stock in and stock out, and to set each item’s full stock when it is added, with input validation.
3. To restrict the available menu options and staff account management according to the login role.

1.4 Scope

The School Canteen Inventory Monitoring System is a Python console-based application for a school canteen. It is used by the canteen manager and the canteen staff in a text-based console on a local computer, one user at a time. Items and accounts are kept in the computer's temporary memory (Python lists) while the program is running.

- **Manager:** logs in, adds new items (the quantity entered becomes the item's full stock), records stock in and stock out, adjusts stock by physical count, deletes items, views the low stock and inventory summary reports, and views, adds, resets the password of, and removes staff accounts.
- **Staff:** logs in, views all items, records stock in and stock out, and views the low stock report.
- **Data handled:** item ID, item name, category (Beverages, Snacks, Rice Meals, Bread & Pastries, Ingredients, or Disposables), quantity, status, and full stock; and the username, password, and role of each account.
- **Boundaries:** the program starts with built-in sample items and accounts, an item is Low Stock when its quantity is 20% or less of its full stock, and login is locked after three failed attempts.

1.5 Limitations

The system is a simple console program with no graphical interface and no online access. It stores items and accounts only in memory, so every change, including new items, stock changes, and added, reset, or removed staff accounts, is lost when the program closes, and the built-in sample items and accounts are loaded again at the next start. It supports one user at a time on one computer and does not support simultaneous use across devices. Login uses basic username and password checking, with passwords stored as plain text and without encryption or advanced authentication. An item's full stock is set once when the item is added and cannot be edited afterward, so changing it requires deleting and re-adding the item. The program does not send notifications or reminders, does not record who changed an item or when, tracks quantities only (no prices, sales amounts, or expiry dates), and does not export or print reports. Low stock alerts appear on the screen during a session only.

1.6 System Requirements

Hardware Requirements

- Processor: AMD Ryzen 5 8500GE
- RAM: 8 GB
- Storage: 238 GB
- Monitor: School desktop monitor
- Input Devices: Keyboard and mouse
- Network: None required

Software Requirements

- Operating System: Windows 11
- Programming Language: Python 3.14.4 (standard library only)
- Development Environment: Visual Studio Code

Functional Requirements

Manager

- The manager can log in with a username and password and see all eleven main menu options.
- The manager can add a new item by entering its name, category, and quantity; the quantity becomes the item’s full stock, and the system assigns the next item ID, the status, and the low stock level at 20% of the full stock.
- The manager can record stock in and stock out, and the system updates the quantity and status after every change.
- The manager can adjust an item’s quantity by physical count and delete an item after confirming.
- The manager can view the inventory summary report (totals, status percentages, units per category, and total units needed to restock).
- The manager can view, add, reset the password of, and remove staff accounts.

Staff

- The staff user can log in and see only the staff menu.
- The staff user can view all items, record stock in, and record stock out.

- The staff user can view the low stock report with the suggested restock amounts.

1.7 Complete Source Code of the School Canteen Inventory Monitoring System

This section shows the complete source code of the two program files, ims.py and login_db.py, in the same order as they appear in the files. Each part of the code comes with an explanation of what it does, including how the colors on the screen are produced. The program is made of only two files: login_db.py keeps the user accounts, and ims.py holds the inventory, the screens, and the main program loop.

Color Guide

The program shows color in the console by using ANSI escape codes. An escape code is a short piece of text that begins with the invisible ESC character (written \033 in Python), followed by [, a number, and the letter m. When the console prints it, the console changes the color or style of every character that comes after it, until a reset code (\033[0m) is printed. The program wraps its text in these codes through one function called paint, so the colors can be switched off in one place (USE_COLOR) if a console shows strange characters.

Name in code	Code	Look	Where it is used
red	\033[91m	Bright red	[ERROR], [LOCKED], Out of Stock status, out-of-stock bar and line
green	\033[92m	Bright green	[SUCCESS], In Stock status, in-stock bar and line, "all items are sufficiently stocked"
yellow	\033[93m	Bright yellow	[ALERT], [WARNING], Low Stock status, menu section labels, restock counts
blue	\033[94m	Bright blue	Divider lines under section headers and table headers
cyan	\033[96m	Bright cyan	Title boxes, [CANCELLED], menu numbers, table headers, section titles
bold	\033[1m	Heavier text	Titles, tags such as [ERROR], table headers, key numbers
dim	\033[2m	Faded text	"Press Enter to continue" messages and thin separator lines
COLOR_RESET	\033[0m	Back to normal	Added after every painted text so colors do not leak to the next line

Status colors follow a traffic-light idea: green means the item is fine, yellow means it is running low, and red means it is gone. The same three colors are used in the stock list, the alerts, and the summary report so the manager learns them once.

1.7.1 ims.py

Part 1 – Imports and settings

The file starts by importing os, which is used to clear the screen, to detect Windows, and to turn on colors in Windows consoles, and login_db, which stores and checks the accounts.

- LOW_STOCK_PERCENT = 20: an item is Low Stock when its quantity is 20% or less of its own full stock.
- MAX_LOGIN_ATTEMPTS = 3: the login is locked after three wrong tries.
- MAX_NAME_LENGTH = 22: the longest item name allowed, so the name fits the 22-character column of the stock list.
- LINE_WIDTH = 75: the width in characters of every title box, divider, and table, so all screens line up.
- USE_COLOR = True: the master switch for colors. Setting it to False prints plain text.
- The os.name == "nt" check detects Windows. Running os.system("") once makes the Windows console understand ANSI color codes.

```
import os

import login_db

LOW_STOCK_PERCENT = 20
MAX_LOGIN_ATTEMPTS = 3
MAX_NAME_LENGTH = 22
LINE_WIDTH = 75

USE_COLOR = True

if USE_COLOR and os.name == "nt":
    os.system("")
```

Part 2 – Color codes

COLOR_CODES is a dictionary that connects a simple color or style name to its escape code, as shown in the Color Guide above. COLOR_RESET is the code that returns the console to normal text. Keeping all codes in one place means a color can be changed here and it changes everywhere.

```
COLOR_CODES = {
    "red": "\033[91m",
    "green": "\033[92m",
    "yellow": "\033[93m",
    "blue": "\033[94m",
    "cyan": "\033[96m",
    "bold": "\033[1m",
    "dim": "\033[2m",
}
```

```
COLOR_RESET = "\033[0m"
```

Part 3 – Inventory data and sample items

inventory is the list that holds all items while the program runs. Each item is a record in this order: [item ID, name, category, quantity, status, full stock]. categories is the list of six allowed categories shown when adding an item. sample_items holds the built-in items in the order [name, category, quantity, full stock], for example a Royal Mismo with 8 on hand out of a full stock of 48. Because the program does not use a file or database, these samples are loaded each time the program starts.

```
inventory = []
categories = ["Beverages", "Snacks", "Rice Meals", "Bread & Pastries",
              "Ingredients", "Disposables"]

sample_items = [
    ["Bottled Water 500ml", "Beverages", 48, 60],
    ["Royal Mismo 200ml", "Beverages", 8, 48],
    ["Coke Mismo 200ml", "Beverages", 25, 48],
    ["Sprite Mismo 200ml", "Beverages", 18, 48],
    ["C2 Apple 230ml", "Beverages", 30, 48],
    ["Zesto Orange Juice", "Beverages", 42, 60],
    ["Milo Sachet", "Beverages", 0, 100],
    ["Nescafe 3-in-1 Sachet", "Beverages", 55, 100],
    ["Piattos Cheese", "Snacks", 32, 48],
    ["Chippy BBQ", "Snacks", 6, 48],
    ["Nova Cheese", "Snacks", 40, 48],
    ["Skyflakes Crackers", "Snacks", 36, 60],
    ["Fita Biscuit", "Snacks", 0, 60],
    ["Lucky Me Pancit Canton", "Snacks", 24, 48],
    ["Boiled Peanuts Pack", "Snacks", 15, 30],
    ["Chicken Adobo w/ Rice", "Rice Meals", 20, 40],
    ["Pork Sinigang w/ Rice", "Rice Meals", 35, 40],
    ["Tapsilog", "Rice Meals", 14, 40],
    ["Hotsilog", "Rice Meals", 28, 40],
    ["Pork Menudo w/ Rice", "Rice Meals", 6, 40],
    ["Pandesal Pack", "Bread & Pastries", 60, 100],
    ["Malunggay Pandesal", "Bread & Pastries", 35, 50],
    ["Ensaymada", "Bread & Pastries", 9, 48],
    ["Spanish Bread", "Bread & Pastries", 22, 48],
    ["Pan de Coco", "Bread & Pastries", 16, 48],
    ["Rice (25kg sack)", "Ingredients", 3, 20],
    ["Cooking Oil 1L", "Ingredients", 10, 48],
    ["Eggs (Tray of 30)", "Ingredients", 6, 30],
    ["Soy Sauce 1L", "Ingredients", 8, 48],
    ["Vinegar 1L", "Ingredients", 9, 48],
    ["Tender Juicy Hotdog", "Ingredients", 45, 100],
    ["Paper Cups", "Disposables", 60, 100],
    ["Plastic Spoon Pack", "Disposables", 7, 50],
    ["Styro Meal Box", "Disposables", 80, 100],
    ["Plastic Straw Pack", "Disposables", 25, 100],
    ["Plastic Labo (Small)", "Disposables", 0, 100],
]
```

Part 4 – Display helpers

These small functions support the screens and the colors.

- clear_screen runs cls on Windows or clear on other systems to wipe the console before each screen.
- pause waits until the user presses Enter. Its message is painted dim so it does not compete with the content.
- paint is the color engine. If USE_COLOR is False it returns the text unchanged. Otherwise it joins the codes of all requested styles, adds the text, and ends with COLOR_RESET. For example, paint("Low", "yellow", "bold") prints bold yellow text.
- status_color returns green for In Stock, yellow for Low Stock, and red for anything else (Out of Stock).
- paint_status first pads the status text to a fixed width and then paints it. Padding comes first because the invisible color codes would otherwise count as characters and break the column alignment.
- make_bar draws a text bar such as [#######.....]. It fills a number of # signs equal to part divided by whole, scaled to a width of 20, and returns an empty bar when the whole is 0 so there is no division by zero.

```
def clear_screen():
    os.system("cls" if os.name == "nt" else "clear")


def pause(message="Press Enter to continue..."):
    input("\n  " + paint(message, "dim"))


def paint(text, *styles):
    if not USE_COLOR:
        return text
    codes = ""
    for style in styles:
        codes += COLOR_CODES[style]
    return codes + text + COLOR_RESET


def status_color(status):
    if status == "In Stock":
        return "green"
    elif status == "Low Stock":
        return "yellow"
    else:
        return "red"


def paint_status(status, width):
    return paint(f"{status:<{width}}", status_color(status))


def make_bar(part, whole, width=20):
    if whole == 0:
        filled = 0
```

```
    else:
        filled = round(part / whole * width)
    return "[" + "#" * filled + "." * (width - filled) + "]"
```

Part 5 – Message tags

Each tag is a ready-made colored label placed in front of a message so the user can tell the kind of message at a glance.

- [ERROR] is bold red for invalid input and failed actions.
- [SUCCESS] is bold green when an action worked.
- [ALERT] is bold yellow for low and out-of-stock warnings.
- [WARNING] is bold yellow. It is defined for future use and is not shown by any screen yet.
- [CANCELLED] is bold cyan when the user answers No to a confirmation.
- [LOCKED] is bold red when too many failed logins close the system.

```
TAG_ERROR = paint("[ERROR]", "red", "bold")
TAG_SUCCESS = paint("[SUCCESS]", "green", "bold")
TAG_ALERT = paint("[ALERT]", "yellow", "bold")
TAG_WARNING = paint("[WARNING]", "yellow", "bold")
TAG_CANCELLED = paint("[CANCELLED]", "cyan", "bold")
TAG_LOCKED = paint("[LOCKED]", "red", "bold")
```

Part 6 – Stock status and calculations

This is the core logic of the system. None of these functions print anything; they only calculate.

- compute_low_limit takes 20% of an item's full stock and drops the decimals. With a full stock of 87 it returns 17, so an item turns Low Stock at 17 or less.
- get_status returns Out of Stock when the quantity is 0, Low Stock when it is at or below the low limit, and In Stock otherwise.
- compute_total_units adds the quantities of all items.
- count_by_status counts how many items have a given status.
- compute_category_total adds the quantities of the items in one category.
- compute_restock_amount returns how many units are needed to get back to full stock (full stock minus quantity, never below 0).
- compute_total_restock adds the restock amounts of all low and out-of-stock items.
- compute_percentage gives part over whole as a percent with one decimal and returns 0.0 when the whole is 0.

```
def compute_low_limit(full_stock):
    return full_stock * LOW_STOCK_PERCENT // 100


def get_status(quantity, full_stock):
    if quantity == 0:
        return "Out of Stock"
    elif quantity <= compute_low_limit(full_stock):
        return "Low Stock"
    else:
        return "In Stock"


def compute_total_units():
    total = 0
    for record in inventory:
        total += record[3]
    return total


def count_by_status(status):
    count = 0
    for record in inventory:
        if record[4] == status:
            count += 1
    return count


def compute_category_total(category):
    total = 0
    for record in inventory:
        if record[2] == category:
            total += record[3]
    return total


def compute_restock_amount(record):
    return max(0, record[5] - record[3])


def compute_total_restock():
    total = 0
    for record in get_low_stock_items():
        total += compute_restock_amount(record)
    return total


def compute_percentage(part, whole):
    if whole == 0:
        return 0.0
    return round(part / whole * 100, 1)
```

Part 7 – Finding, adding, changing, and deleting items

These functions work directly on the inventory list.

- find_item searches the list for an item ID and returns the record, or None when not found.
- name_exists checks for a duplicate name and ignores upper and lower case, so "juice" and "Juice" count as the same.

- generate_item_id starts from 1000 and returns the highest ID plus one, so IDs are always unique and start at 1001.
- add_item creates a record with a new ID and the status computed from the quantity and full stock, then appends it to the inventory.
- change_quantity sets a new quantity and recomputes the status against the item's own full stock.
- delete_item removes a record from the list.
- get_low_stock_items returns every item that is Low Stock or Out of Stock.
- load_sample_items adds every item of sample_items to the inventory at program start.

```
def find_item(item_id):
    for record in inventory:
        if record[0] == item_id:
            return record
    return None


def name_exists(item_name):
    for record in inventory:
        if record[1].lower() == item_name.lower():
            return True
    return False


def generate_item_id():
    highest = 1000
    for record in inventory:
        if record[0] > highest:
            highest = record[0]
    return highest + 1


def add_item(item_name, category, quantity, full_stock):
    record = [generate_item_id(), item_name, category, quantity,
              get_status(quantity, full_stock), full_stock]
    inventory.append(record)
    return record


def change_quantity(record, new_quantity):
    record[3] = new_quantity
    record[4] = get_status(new_quantity, record[5])


def delete_item(record):
    inventory.remove(record)


def get_low_stock_items():
    low_items = []
    for record in inventory:
        if record[4] == "Low Stock" or record[4] == "Out of Stock":
            low_items.append(record)
    return low_items
```

```
def load_sample_items():
    for sample in sample_items:
        add_item(sample[0], sample[1], sample[2], sample[3])
```

Part 8 – Menu options by role

get_menu_options is what separates the manager from the staff. It returns a list of options, and each option holds three things: an action name used by the program, the text shown on screen, and the section heading it is grouped under. The Manager gets eleven options, including adding and deleting items, physical count, the summary report, and staff account management. Any other role (Staff) gets only six options, so manager-only functions are never shown to them.

```
def get_menu_options(role):
    if role == "Manager":
        return [
            ["view", "View All Items", "INVENTORY"],
            ["add", "Add New Item", "INVENTORY"],
            ["stock_in", "Stock In (Delivery)", "INVENTORY"],
            ["stock_out", "Stock Out (Sold / Used)", "INVENTORY"],
            ["count", "Physical Count (Adjust Stock)", "INVENTORY"],
            ["delete", "Delete Item", "INVENTORY"],
            ["low", "Low Stock Report", "REPORTS"],
            ["summary", "Inventory Summary Report", "REPORTS"],
            ["accounts", "Manage Staff Accounts", "ADMINISTRATION"],
            ["logout", "Log Out", "SESSION"],
            ["exit", "Exit System", "SESSION"],
        ]
    else:
        return [
            ["view", "View All Items", "INVENTORY"],
            ["stock_in", "Stock In (Delivery)", "INVENTORY"],
            ["stock_out", "Stock Out (Sold / Used)", "INVENTORY"],
            ["low", "Low Stock Report", "REPORTS"],
            ["logout", "Log Out", "SESSION"],
            ["exit", "Exit System", "SESSION"],
        ]
```

Part 9 – Input readers

These three functions keep asking until the user types something valid, which is how the system validates every entry.

- read_text repeats until the text is not empty.
- read_number repeats until the text is a whole number that is at least the given minimum. This blocks letters, decimals, negative numbers, and blanks.
- read_menu_choice repeats until the number is between 1 and the number of options.

```
def read_text(prompt):
    text = input(prompt).strip()
    while text == "":
        print(f"  {TAG_ERROR} Input cannot be empty.")
        text = input(prompt).strip()
    return text
```

```
def read_number(prompt, minimum):
    text = input(prompt).strip()
    while not text.isdigit() or int(text) < minimum:
        print(f"  {TAG_ERROR} Please enter a whole number "
              f"(minimum {minimum}).")
        text = input(prompt).strip()
    return int(text)


def read_menu_choice(prompt, total_options):
    choice = input(prompt).strip()
    while not choice.isdigit() or not 1 <= int(choice) <= total_options:
        print(f"\n  {TAG_ERROR} Invalid option. "
              f"Please enter 1 to {total_options}.")
        choice = input(prompt).strip()
    return int(choice)
```

Part 10 – Screen layout and pickers

These functions draw the repeating parts of the screens and ask the user to pick things.

- print_title draws a cyan box with box-drawing characters. The title is bold and centered, and an optional subtitle is placed below a second cyan line.
- print_section_header prints a blue line, a bold cyan title marked with a small block, and another blue line.
- print_line prints a thin dim line used to separate groups of text.
- choose_category lists the six categories with numbers and keeps asking until the number is on the list.
- pick_item first checks that the inventory is not empty, then asks for an item ID and returns the record, or shows [ERROR] when the ID does not exist.

```
def print_title(title, subtitle=""):
    inner = LINE_WIDTH - 2
    print("\n" + paint("╔" + "═" * inner + "╗", "cyan"))
    print(paint("║", "cyan") + paint(title.center(inner), "bold")
          + paint("║", "cyan"))
    if subtitle != "":
        print(paint("╠" + "═" * inner + "╣", "cyan"))
        print(paint("║", "cyan") + subtitle.center(inner)
              + paint("║", "cyan"))
    print(paint("╚" + "═" * inner + "╝", "cyan"))


def print_section_header(title):
    print("\n" + paint("─" * LINE_WIDTH, "blue"))
    print(paint(f" ▌ {title}", "bold", "cyan"))
    print(paint("─" * LINE_WIDTH, "blue"))


def print_line(char="─"):
    print(paint(char * LINE_WIDTH, "dim"))


def choose_category():
    print("\n  Select a category:")
    for i in range(len(categories)):
```

```
        print(f"    [{i + 1}] {categories[i]}")
    choice = read_number("  Enter category number: ", 1)
    while choice > len(categories):
        print(f"  {TAG_ERROR} Category number is not on the list.")
        choice = read_number("  Enter category number: ", 1)
    return categories[choice - 1]


def pick_item(prompt):
    if len(inventory) == 0:
        print(f"  {TAG_ERROR} The inventory is empty. Add an item first.")
        return None
    item_id = read_number(prompt, 0)
    record = find_item(item_id)
    if record is None:
        print(f"  {TAG_ERROR} Item ID not found!")
    return record
```

Part 11 – Staff account input

read_new_username keeps asking until the username has at least 3 characters, has no spaces, and is not already used (checked without regard to upper or lower case). read_new_password keeps asking until the password has at least 4 characters. The minimum lengths come from login_db.

```
def read_new_username():
    username = read_text("  New username: ")
    while (len(username) < login_db.MIN_USERNAME_LENGTH
           or " " in username
           or login_db.username_exists(username)):
        if len(username) < login_db.MIN_USERNAME_LENGTH or " " in username:
            print(f"  {TAG_ERROR} Username needs "
                  f"{login_db.MIN_USERNAME_LENGTH}+ "
                  f"characters and no spaces.")
        else:
            print(f"  {TAG_ERROR} That username is already taken.")
        username = read_text("  New username: ")
    return username


def read_new_password(prompt):
    password = read_text(prompt)
    while len(password) < login_db.MIN_PASSWORD_LENGTH:
        print(f"  {TAG_ERROR} Password needs at least "
              f"{login_db.MIN_PASSWORD_LENGTH} characters.")
        password = read_text(prompt)
    return password
```

Part 12 – Main menu and stock alert

display_menu shows the main menu. The title box has the logged-in username and role in its subtitle. The options are grouped by their section heading, which is printed in bold yellow, and each option number is shown in cyan.

show_stock_alert prints a message after a stock change. For Low Stock it shows the quantity left and the suggested restock amount. For Out of Stock it shows OUT OF STOCK in bold red with the restock amount. In Stock items print nothing.

```
def display_menu(username, role, options):
    print_title("SCHOOL CANTEEN INVENTORY MONITORING SYSTEM",
                f"MAIN MENU  |  {username}  ({role})")
    last_section = ""
    for i in range(len(options)):
        if options[i][2] != last_section:
            last_section = options[i][2]
            print("\n  " + paint(f"■ {last_section}", "bold", "yellow"))
        print(f"     {paint(f'[{i + 1:>2}]', 'cyan')} {options[i][1]}")
    print("\n" + paint("=" * LINE_WIDTH, "cyan"))


def show_stock_alert(record):
    if record[4] == "Low Stock":
        print(f"  {TAG_ALERT} Low stock! '{record[1]}' has only "
              f"{record[3]} left. "
              f"Suggested restock: {compute_restock_amount(record)} "
              f"units.")
    elif record[4] == "Out of Stock":
        print(f"  {TAG_ALERT} '{record[1]}' is "
              f"{paint('OUT OF STOCK!', 'red', 'bold')} "
              f"Suggested restock: {compute_restock_amount(record)} "
              f"units.")
```

Part 13 – Stock list and reports

These functions are the read-only screens.

- display_all_items prints the stock list as a table. The header is bold cyan with a blue divider. The STOCK column shows the quantity over the full stock, such as 48/60, and the status is painted with its status color.
- display_low_stock_report lists only low and out-of-stock items with the units needed to restock. When nothing is low, a green message is shown instead. The count of items needing restock is bold yellow.
- display_summary_report shows the total records and total units, then the In Stock, Low Stock, and Out of Stock counts with percentages and a bar, painted green, yellow, and red. It ends with the units per category and the total units needed to restock.
- display_staff_accounts lists staff usernames with cyan numbers.

```
def display_all_items():
    print_title("CURRENT CANTEEN STOCK LIST")
    if len(inventory) == 0:
        print(" No items recorded yet.")
    else:
        header = (f"{'ID':<6} | {'ITEM NAME':<22} | {'CATEGORY':<16} | "
                  f"{'STOCK':>7} | {'STATUS':<12}")
        print(paint(header, "bold", "cyan"))
        print(paint("-" * LINE_WIDTH, "blue"))
        for record in inventory:
            stock_text = f"{record[3]}/{record[5]}"
            print(f"{record[0]:<6} | {record[1]:<22} | {record[2]:<16} | "
                  f"{stock_text:>7} | {paint_status(record[4], 12)}")
    print(paint("=" * LINE_WIDTH, "cyan"))
```

```
def display_low_stock_report():
    low_items = get_low_stock_items()
    print_title("LOW STOCK ALERT REPORT")
    if len(low_items) == 0:
        print(paint(" All items are sufficiently stocked.", "green"))
    else:
        header = (f"{'ID':<6} | {'ITEM NAME':<22} | {'QTY':>5} | "
                  f"{'STATUS':<12} | {'RESTOCK':>8}")
        print(paint(header, "bold", "cyan"))
        print(paint("-" * LINE_WIDTH, "blue"))
        for record in low_items:
            restock = compute_restock_amount(record)
            print(f"{record[0]:<6} | {record[1]:<22} | {record[3]:>5} | "
                  f"{paint_status(record[4], 12)} | {restock:>8}")
        print(paint("-" * LINE_WIDTH, "blue"))
        print(f" Items needing restock: "
              f"{paint(str(len(low_items)), 'bold', 'yellow')}")
    print(paint("=" * LINE_WIDTH, "cyan"))


def display_summary_report():
    total_items = len(inventory)
    in_stock = count_by_status("In Stock")
    low_stock = count_by_status("Low Stock")
    out_stock = count_by_status("Out of Stock")
    print_title("INVENTORY SUMMARY REPORT")
    print(f" Total item records  : {total_items}")
    print(f" Total units in stock: {compute_total_units()}")
    print_line()
    print(paint(f" In Stock    : {in_stock:>3} item(s) "
                f"({compute_percentage(in_stock, total_items)}%)",
                "green"))
    print("   " + paint(make_bar(in_stock, total_items), "green"))
    print(paint(f" Low Stock   : {low_stock:>3} item(s) "
                f"({compute_percentage(low_stock, total_items)}%)",
                "yellow"))
    print("   " + paint(make_bar(low_stock, total_items), "yellow"))
    print(paint(f" Out of Stock: {out_stock:>3} item(s) "
                f"({compute_percentage(out_stock, total_items)}%)",
                "red"))
    print("   " + paint(make_bar(out_stock, total_items), "red"))
    print_line()
    print(paint(" Units per category:", "bold"))
    for category in categories:
        print(f"   {category:<20}: {compute_category_total(category):>5}")
    print_line()
    print(f" Total units needed to restock: "
          f"{paint(str(compute_total_restock()), 'bold', 'yellow')}")
    print(paint("=" * LINE_WIDTH, "cyan"))


def display_staff_accounts():
    staff_list = login_db.get_staff_usernames()
    print_section_header("STAFF ACCOUNTS")
    if len(staff_list) == 0:
        print(" No staff accounts yet.")
    else:
        for i in range(len(staff_list)):
            print(f"  {paint(f'[{i + 1}]', 'cyan')} {staff_list[i]}")
    print_line()
```

Part 14 – Inventory actions

Each handler is one menu action. All of them use the input readers, so every entry is validated.

- handle_add_item asks for a name (not empty, at most 22 characters, not a duplicate), a category, and a quantity of at least 1. The quantity is saved as both the current quantity and the full stock, then the program shows the full stock and the number at which the low stock alert will come (20%).
- handle_stock_in adds the quantity received to the current quantity and shows the old and new quantity with the new status.
- handle_stock_out removes the quantity sold or used, but refuses an amount larger than the quantity on hand. It shows an alert if the item became low or out of stock.
- handle_physical_count replaces the system quantity with the counted quantity, which may be 0, and shows an alert if needed.
- handle_delete_item asks for confirmation. Only Y deletes the item; anything else shows [CANCELLED].

```
def handle_add_item():
    print_section_header("ADD NEW ITEM")
    item_name = read_text("  Enter item name: ")
    while len(item_name) > MAX_NAME_LENGTH or name_exists(item_name):
        if len(item_name) > MAX_NAME_LENGTH:
            print(f"  {TAG_ERROR} Name must be {MAX_NAME_LENGTH} characters "
                  f"or less.")
        else:
            print(f"  {TAG_ERROR} Item already exists. "
                  f"Use Stock In instead.")
        item_name = read_text("  Enter item name: ")
    category = choose_category()
    quantity = read_number("  Enter quantity (this is the full stock): ", 1)
    record = add_item(item_name, category, quantity, quantity)
    print(f"\n  {TAG_SUCCESS} Item added: {record[1]} (ID: {record[0]}) "
          f"- Status: {paint(record[4], status_color(record[4]))}")
    print(f"  Full stock: {record[5]} | Low stock alert at "
          f"{compute_low_limit(record[5])} or less ({LOW_STOCK_PERCENT}%)")
    show_stock_alert(record)


def handle_stock_in():
    print_section_header("STOCK IN (DELIVERY)")
    record = pick_item("  Enter Item ID: ")
    if record is None:
        return
    print(f"  Selected: {record[1]} | Quantity: {record[3]}/{record[5]} "
          f"({paint(record[4], status_color(record[4]))})")
    amount = read_number("  Enter quantity received: ", 1)
    old_quantity = record[3]
    change_quantity(record, old_quantity + amount)
    print(f"\n  {TAG_SUCCESS} {record[1]}: {old_quantity} -> {record[3]} "
          f"({paint(record[4], status_color(record[4]))})")
```

```
def handle_stock_out():
    print_section_header("STOCK OUT (SOLD / USED)")
    record = pick_item("  Enter Item ID: ")
    if record is None:
        return
    print(f"  Selected: {record[1]} | Quantity: {record[3]}/{record[5]} "
          f"({paint(record[4], status_color(record[4]))})")
    amount = read_number("  Enter quantity sold/used: ", 1)
    if amount > record[3]:
        print(f"  {TAG_ERROR} Cannot remove {amount}. "
              f"Only {record[3]} in stock.")
        return
    old_quantity = record[3]
    change_quantity(record, old_quantity - amount)
    print(f"\n  {TAG_SUCCESS} {record[1]}: {old_quantity} -> {record[3]} "
          f"({paint(record[4], status_color(record[4]))})")
    show_stock_alert(record)


def handle_physical_count():
    print_section_header("PHYSICAL COUNT (ADJUST STOCK)")
    record = pick_item("  Enter Item ID: ")
    if record is None:
        return
    print(f"  Selected: {record[1]} | System quantity: {record[3]} "
          f"({paint(record[4], status_color(record[4]))})")
    new_quantity = read_number("  Enter the counted quantity: ", 0)
    old_quantity = record[3]
    change_quantity(record, new_quantity)
    print(f"\n  {TAG_SUCCESS} {record[1]}: {old_quantity} -> {record[3]} "
          f"({paint(record[4], status_color(record[4]))})")
    show_stock_alert(record)


def handle_delete_item():
    print_section_header("DELETE ITEM")
    record = pick_item("  Enter Item ID to delete: ")
    if record is None:
        return
    confirm = input(f"  Delete '{record[1]}'? (Y/N): ").strip().lower()
    if confirm == "y":
        delete_item(record)
        print(f"\n  {TAG_SUCCESS} '{record[1]}' was removed from the "
              f"inventory.")
    else:
        print(f"\n  {TAG_CANCELLED} No changes were made.")
```

Part 15 – Staff account actions

These functions are used only by the manager.

- pick_staff_account shows the staff list and returns the chosen username.
- handle_add_staff creates a Staff account with a validated username and password.
- handle_reset_password changes the password of the chosen staff member.
- handle_remove_staff removes a staff account after a Y/N confirmation.
- handle_manage_accounts is a small sub-menu loop with five choices (view, add, reset, remove, back). It clears the screen, runs the chosen action, and waits for Enter

until the manager chooses Back.

```
def pick_staff_account():
    staff_list = login_db.get_staff_usernames()
    if len(staff_list) == 0:
        print(f"  {TAG_ERROR} There are no staff accounts.")
        return None
    display_staff_accounts()
    number = read_number("  Enter staff number: ", 1)
    if number > len(staff_list):
        print(f"  {TAG_ERROR} Staff number is not on the list.")
        return None
    return staff_list[number - 1]


def handle_add_staff():
    print_section_header("ADD STAFF ACCOUNT")
    username = read_new_username()
    password = read_new_password("  New password: ")
    login_db.add_staff_account(username, password)
    print(f"\n  {TAG_SUCCESS} Staff account '{username}' created.")


def handle_reset_password():
    print_section_header("RESET STAFF PASSWORD")
    username = pick_staff_account()
    if username is None:
        return
    password = read_new_password("  New password: ")
    login_db.reset_password(username, password)
    print(f"\n  {TAG_SUCCESS} Password for '{username}' was changed.")


def handle_remove_staff():
    print_section_header("REMOVE STAFF ACCOUNT")
    username = pick_staff_account()
    if username is None:
        return
    confirm = input(f"  Remove '{username}'? (Y/N): ").strip().lower()
    if confirm == "y":
        login_db.remove_account(username)
        print(f"\n  {TAG_SUCCESS} Staff account '{username}' was removed.")
    else:
        print(f"\n  {TAG_CANCELLED} No changes were made.")


def handle_manage_accounts():
    while True:
        clear_screen()
        print_title("MANAGE STAFF ACCOUNTS")
        print(f"     {paint('[1]', 'cyan')} View Staff Accounts")
        print(f"     {paint('[2]', 'cyan')} Add Staff Account")
        print(f"     {paint('[3]', 'cyan')} Reset Staff Password")
        print(f"     {paint('[4]', 'cyan')} Remove Staff Account")
        print(f"     {paint('[5]', 'cyan')} Back to Main Menu")
        print(paint("=" * LINE_WIDTH, "cyan"))
        choice = read_menu_choice("Select an option (1-5): ", 5)
        if choice == 5:
            return
        clear_screen()
        if choice == 1:
            display_staff_accounts()
        elif choice == 2:
            handle_add_staff()
        elif choice == 3:
```

```
            handle_reset_password()
        else:
            handle_remove_staff()
        pause("Press Enter to go back...")
```

Part 16 – Login, main loop, and program start

This is the part that ties everything together.

- login shows the login screen and allows up to three tries. It returns the username and role when the credentials are correct, or an empty list after three failures.
- run_menu gets the menu for the role, then repeats: clear the screen, show the menu, read a choice, and run the matching action. The accounts option runs its own sub-menu and returns straight to the menu. Log Out returns True so the login screen appears again, and Exit System returns False to close the program.
- main loads the sample items once, then keeps showing the login screen until the login is locked or the user exits. When the login is locked, [LOCKED] is shown and the program closes.
- The last two lines make main run only when the file is started directly (python ims.py).

```
def login():
    clear_screen()
    print_title("SCHOOL CANTEEN INVENTORY MONITORING SYSTEM",
                "Authorized Personnel Login")
    attempts = 0
    while attempts < MAX_LOGIN_ATTEMPTS:
        username = input("  Username: ").strip()
        password = input("  Password: ").strip()
        role = login_db.check_credentials(username, password)
        if role != "":
            print(f"\n  {TAG_SUCCESS} Welcome, {username} ({role})!")
            return [username, role]
        attempts += 1
        remaining = MAX_LOGIN_ATTEMPTS - attempts
        print(f"\n  {TAG_ERROR} Invalid username or password. "
              f"Attempts left: {remaining}\n")
    return []


def run_menu(username, role):
    options = get_menu_options(role)
    while True:
        clear_screen()
        display_menu(username, role, options)
        choice = read_menu_choice(f"Select an option (1-{len(options)}): ",
                                  len(options))
        action = options[choice - 1][0]
        clear_screen()
        if action == "view":
            display_all_items()
        elif action == "add":
            handle_add_item()
        elif action == "stock_in":
```

```
            handle_stock_in()
        elif action == "stock_out":
            handle_stock_out()
        elif action == "count":
            handle_physical_count()
        elif action == "delete":
            handle_delete_item()
        elif action == "low":
            display_low_stock_report()
        elif action == "summary":
            display_summary_report()
        elif action == "accounts":
            handle_manage_accounts()
            continue
        elif action == "logout":
            print("\n  Logged out successfully.")
            pause("Press Enter to go to the login screen...")
            return True
        else:
            print("\n  Exiting system. Goodbye!")
            return False
        pause("Press Enter to return to the main menu...")


def main():
    load_sample_items()
    keep_running = True
    while keep_running:
        session = login()
        if len(session) == 0:
            print(f"  {TAG_LOCKED} Too many failed attempts. System closed.")
            keep_running = False
        else:
            keep_running = run_menu(session[0], session[1])


if __name__ == "__main__":
    main()
```

1.7.2 login_db.py

Complete file

login_db.py is the account storage. It keeps accounts in three parallel lists, which means the item at the same position in each list belongs to the same account. The program starts with two accounts: one Manager and one Staff. Like the inventory, accounts exist only in memory while the program runs.

- MIN_USERNAME_LENGTH = 3 and MIN_PASSWORD_LENGTH = 4 are the minimum lengths used by ims.py when creating staff accounts.
- check_credentials looks for a position where both username and password match and returns that account's role, or an empty text when there is no match. ims.py uses the role to decide which menu to show.
- find_account_position returns the list position of a username, or -1 when not found.

- username_exists checks if a username is already taken, ignoring upper and lower case.
- get_staff_usernames returns only the usernames whose role is Staff, so the manager account is never shown in the staff list and cannot be removed from the menu.
- add_staff_account adds a username, password, and the role Staff to the three lists.
- reset_password finds the account and replaces its password.
- remove_account removes the username, password, and role at the same position from the three lists so they stay matched.

```
MIN_USERNAME_LENGTH = 3
MIN_PASSWORD_LENGTH = 4

usernames = ["gwapong_manager", "gwapangastaff"]
passwords = ["passwordsagwapo", "passwordsagwapa"]
roles = ["Manager", "Staff"]


def check_credentials(username, password):
    for i in range(len(usernames)):
        if usernames[i] == username and passwords[i] == password:
            return roles[i]
    return ""


def find_account_position(username):
    for i in range(len(usernames)):
        if usernames[i] == username:
            return i
    return -1


def username_exists(username):
    for name in usernames:
        if name.lower() == username.lower():
            return True
    return False


def get_staff_usernames():
    staff_list = []
    for i in range(len(usernames)):
        if roles[i] == "Staff":
            staff_list.append(usernames[i])
    return staff_list


def add_staff_account(username, password):
    usernames.append(username)
    passwords.append(password)
    roles.append("Staff")


def reset_password(username, new_password):
    position = find_account_position(username)
    passwords[position] = new_password

def remove_account(username):
```

```
position = find_account_position(username)
usernames.pop(position)
passwords.pop(position)
roles.pop(position)
```

References

Shabani, A., Maroti, G., de Leeuw, S., & Dullaert, W. (2021). Inventory record inaccuracy and store-level performance. *International Journal of Production Economics, 235*, Article 108111. https://doi.org/10.1016/j.ijpe.2021.108111

Sison, P. D. T., Oreiro, K. P., Camalit, J. C. Z., & Ng, G. C. (2019). A problem oriented approach to implementing an inventory and point-of-sale system for Company KCP. *Innovatus, 2*(1), 95–100. https://doi.org/10.5281/zenodo.4484959